AMRITA VISHWA VIDYAPEETHAM

AHEAD Online

Object Oriented Programming Using Java

Dr. MG Thushara
Department of Computer Science
Amrita Vishwa Vidyapeetham

Objectives

- ✓ Strings in Java.

String Handling

- ✓ Java implements strings as objects of type **String**.
- ✓ String is probably the most used class in Java's class library.
- ✓ Every string you create is an object of **type String**.
- ✓ Java includes **extensive string-handling** capabilities that are both powerful and easy to use.
- ✓ Java has methods to **compare** two strings, **search** for a substring, **concatenate** two strings, and **change** the **case** of letters within a string.

String Handling

- ✓ Objects of type String are **immutable**; once a String object is created, its contents cannot be altered.
- ✓ For modifying strings Java provides two options, **StringBuffer** and **StringBuilder**.
  - ✓ *String myString = "this is a test";// Create a string*
  - ✓ *System.out.println(myString); //Display the string*

String Constructor

- ✓ Creating an empty string

  String s = new String();

- ✓ Create a String initialized by an array of characters

  String(char chars[ ])

E.g.

char chars[] = { 'a', 'b', 'c' };

String S = new String(chars);

S is initialized with "abc".

String Constructor

String(char chars[ ], int startIndex, int numChars)

- ✓ *startIndex* specifies the index at which the subrange begins, and *numChars* specifies the number of characters to use.

  char chars[] = { 'a', 'b', 'c', 'd', 'e', 'f' };

  String s = new String(chars, 2, 3);

  This initializes s with the characters cde.

- ✓ We can construct a String object that contains the same character sequence as another String object.

  String(String strObj);

String Length

- ✓ The length of a string is the number of characters that it contains.
- ✓ To obtain this value, call the **length( )** method,
- ✓ The following fragment prints "3", since there are three characters in the string **s**:

```
char chars[] = { 'a', 'b', 'c' };
String s = new String(chars);
System.out.println(s.length());
```

String Concatenation

- ✓ Java uses + operator for string **concatenation**.

```
String age = "9";
String s = "He is " + age + " years old.";
System.out.println(s);
```

- ✓ Concatenation with different types.

```
int age = 9;
String s = "He is " + age + " years old.";
System.out.println(s);
```

String Conversion

- ✓ **valueOf()** converts data into its string representation.
- ✓ To represent any object as a string, toString() method is used.

String toString( )

Character Extraction

- ✓ **charAt( )** will extract a single character from a String.
  - ✓ *char charAt(int where)*, where is the index(nonnegative) of the character that you want to obtain.

```
char ch;
ch = "abc".charAt(1); //assigns the value "b" to ch.
```

- ✓ **getChars( )** will extract more than one character from a string.

  void getChars(int sourceStart, int sourceEnd, char target[ ], int targetStart)

  sourceStart - Index of the beginning of the substring, sourceEnd - an index that is one past the end of the desired substring, target - array that will receive the characters . The index within target at which the substring will be copied is passed in targetStart.

Character Extraction

```
class getCharsDemo {
        public static void main(String args[]) {
                String s = "This is a demo of the getChars method.";
                int start = 10;
                int end = 14;
                char buf[] = new char[end - start];
                s.getChars(start, end, buf, 0);
                System.out.println(buf);
        }
}
```

Output: demo

Character Extraction

- ✓ **getBytes( )** is an alternative to **getChars( )** that stores the characters in an array of bytes.

  byte[ ] getBytes( )

- ✓ **toCharArray( )** convert all the characters in a String object into a character array.

  char[ ] toCharArray( )

String Comparison

- ✓ **equals( )** comparing two strings for equality.

  boolean equals(Object str)

- ✓ **equalsIgnoreCase( )** to perform a comparison that ignores case differences.
  - ✓ When it compares two strings, it considers A-Z to be the same as a-z.

  boolean equalsIgnoreCase(String str)

  Note: **. The == operator compares two object references to see whether they refer to the same instance.**

String Comparison

- ✓ **compareTo( )**

  int compareTo(String str)

Value	Meaning
Less than zero	The invoking string is less than *str*.
Greater than zero	The invoking string is greater than *str*.
Zero	The two strings are equal.

- ✓ **compareToIgnoreCase( )** ignores the case.

  int compareToIgnoreCase(String str)

Searching Strings

- ✓ ***indexOf( )*** Searches for the first occurrence of a character or substring.
- ✓ ***lastIndexOf( )*** Searches for the last occurrence of a character or substring

Takeaways

- ✓ String class in Java.
- ✓ Different operations in string class and corresponding methods.

Reference

[1] Herbert Schildt, *"Java: The Complete Reference, Eleventh Edition", Oracle 2018*